

Function, Basic Module, and Package

PTP Program



Training Guidelines

Persyaratan dan Ekspektasi untuk Peserta Pelatihan:

1. **Prerequisite:** Peserta diharapkan sudah memiliki pengetahuan dasar tipe data dan looping.
2. **Active Participation:** Peserta harus terlibat aktif dengan materi pelatihan, mencatat poin-poin penting dan penjelasan yang diberikan oleh instruktur.
3. **Clear Instruction:** Instruktur bertanggung jawab untuk menyampaikan materi pelatihan dengan cara yang jelas dan ringkas, memastikan semua peserta memahami topik yang dibahas.
4. **Hands-on Practice:** Sebagian besar pelatihan akan melibatkan latihan praktis yang dilakukan menggunakan **Google Colab (Jupyter Notebook)**. Anda dapat mengakses Google Colab yang tersedia di slide terakhir.

List Of Contents

- Basic Python Function
- Function Arguments
- Anonymous Function
- Basic Module & Packages
- PIP

Function

Basic Python Function

Fungsi adalah sekumpulan instruksi yang dirancang untuk melakukan suatu tugas tertentu. Fungsi memungkinkan kita untuk menulis program yang lebih modular, terstruktur, dan mudah dipelihara. Dengan menggunakan fungsi, kita bisa membagi program yang besar menjadi bagian-bagian yang lebih kecil dan lebih mudah dipahami. Berikut adalah manfaat fungsi:

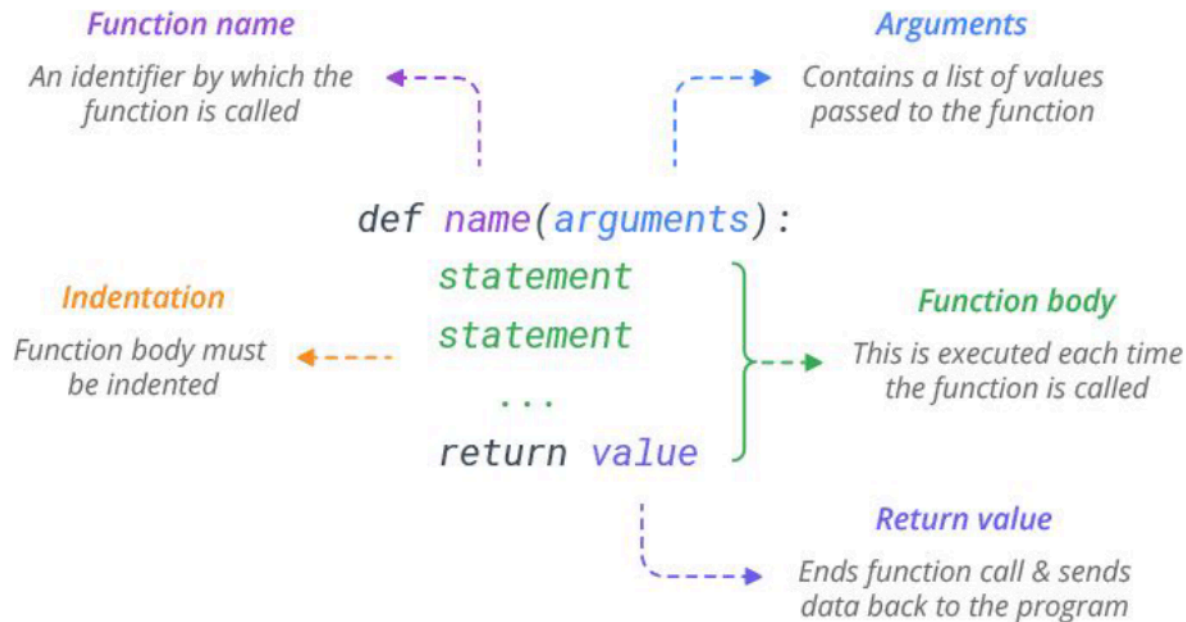
1. **Pengorganisasian Kode:** Fungsi membantu mengorganisir kode sehingga lebih terstruktur.
2. **Reusability:** Fungsi dapat digunakan kembali di berbagai bagian program tanpa harus menulis ulang kode yang sama.
3. **Pemeliharaan:** Kode yang terorganisir dengan fungsi lebih mudah diperbaiki dan ditingkatkan karena setiap fungsi memiliki tujuan spesifik.

Untuk mendeklarasikan sebuah fungsi, sintaksnya adalah:

```
def function_name(arguments):  
    # function body  
  
    return
```

Basic Python Function

Defining a Function



Courtesy:

<https://www.learnbyexample.org/python-functions/>

Basic Python Function

Calling Function

```
# Defining a function
def pow_func(num, pow):
    return num**pow

#Calling the function
print("2^4 =",pow_func(2,4))
```

Misalkan kamu telah mendefinisikan sebuah fungsi untuk menghitung eksponensial, yaitu angka yang dipangkatkan dengan angka lainnya.

Kemudian, untuk menggunakan fungsi yang telah kamu buat, kamu cukup memanggil nama fungsi `pow_func` diikuti dengan tanda kurung dan memasukkan nilai yang ingin kamu ganti pada `num` dan `pow`.

Basic Python Function

Global vs Local Variable

```
c = 1 #Global variable
def func(x1):
    L = x1**2 + c #, L -> Local variable
    print(L)
```

L

```
-----
NameError                                Traceback (most recent call last)
<ipython-input-18-a9dfae9159c1> in <module>
----> 1 L

NameError: name 'L' is not defined
```

Dalam fungsi Python, ada konsep bahwa jika kamu mendefinisikan variabel di dalam fungsi, kamu tidak dapat memanggil variabel tersebut di luar fungsi.

Variabel-variabel ini disebut **Local Variable**. Namun, jika kamu memiliki variabel yang ada di kode umum (di luar fungsi) dan bisa digunakan di dalam fungsi, variabel-variabel tersebut disebut **Global Variable**.

Jika kamu memanggil variabel L di luar fungsi, maka akan muncul error yang mengatakan bahwa variabel tersebut tidak didefinisikan. Jadi, L adalah **Local Variable** yang hanya ada di dalam fungsi tersebut.

Function Arguments

Required, Keyword, and Default Argument

Argumen adalah nilai yang diterima oleh sebuah fungsi. Ada empat jenis argumen yang dapat diterima, yaitu argumen wajib, argumen kata kunci, argumen default, dan argumen panjang variabel.

Required Argument

Argumen yang harus diisi saat fungsi dipanggil.

```
def func(a, b, c=1, d=3):  
    return a+b+c+d
```

Default Argument

Argumen yang sudah diisi dengan nilai default, dan pengisian argumen tersebut bersifat opsional saat fungsi dipanggil.

Potitional Argument

1 dan 2 masing-masing milik argumen a dan b. Nilai-nilai tersebut akan diisi ke dalam argumen sesuai dengan urutannya.

```
func(1, 2, d=3)
```

Keyword Argument

Untuk menggunakan argumen kata kunci (keyword argument), Anda harus menyebutkan argumen tertentu yang ingin diisi dengan nilai. Urutannya tidak penting.

Function Arguments

Variable-Length Argument - Non Keyword

Argumen dengan panjang variabel adalah jenis argumen yang unik dalam fungsi Python. Argumen ini menerima input data yang tidak terbatas. Oleh karena itu, input tersebut akan disimpan dalam Tuple (untuk argumen non-kata kunci yang dilambangkan dengan ***args**) atau Dictionary (untuk argumen kata kunci yang dilambangkan dengan ****kwargs**).

```
def multiplier(*num):  
    prod = 1  
    #initialize prod variable with zero  
    for i in num:  
        prod = prod * i  
  
    print("Product:",prod)  
  
multiplier(3,5)      #Product: 15  
multiplier(1,2,4)    #Product: 8  
multiplier(2,2,6,7)  #Product(168)
```

***args** dapat diganti dengan nama variabel lain seperti ***num** pada contoh di atas.

Function Arguments

Variable-Length Argument - Keyword

```
def daftar_nama(sekolah, **nama_peserta):
    print(f"Peserta didik sekolah {sekolah}:")
    i = 0
    for nama in nama_peserta.values():
        i += 1
        print(f"{i}. {nama}")
    print(nama_peserta)

daftar_nama("Hacktiv8",siswa1="A",siswa2="B",siswa3="C",siswa4="D")

...
_Output_

Peserta didik sekolah Hacktiv8:
1. A
2. B
3. C
4. D
{'siswa1': 'A', 'siswa2': 'B', 'siswa3': 'C', 'siswa4': 'D'}
...
```

"Hacktiv8" akan dimasukkan ke dalam variabel sekolah, dan "A", "B", "C", "D" akan menjadi nilai dari nama_peserta, yang merupakan sebuah kamus (dictionary) yang terdiri dari empat kata kunci sesuai dengan input yang diterima oleh fungsi.

Introduction to

Anonymous Function

Fungsi anonim adalah fungsi yang tidak memiliki nama saat didefinisikan. Fungsi ini menggunakan kata kunci "Lambda" untuk mendefinisikan fungsi, bukan "def". Fungsi ini dapat diterapkan pada variabel, pandas apply, dan masih banyak lagi.

Gunakan format `lambda arg1, arg2 : ekspresi` untuk mendefinisikan fungsi tersebut.

```
#Define a function
penjumlahan = lambda x1, x2: x1 + x2

#Calling the function
penjumlahan(2,4) #Output: 6
```

Introduction to

Basic Module & Packages

Modul adalah sebuah file Python sederhana yang berisi kumpulan fungsi dan variabel global, dengan ekstensi file '.py'. Modul ini merupakan file yang dapat dieksekusi, dan untuk mengorganisir semua modul, kita memiliki konsep yang disebut Paket dalam Python.

Ada beberapa keuntungan dalam memodularisasi kode pada aplikasi besar:

- Kesederhanaan
- Pemeliharaan
- Penggunaan kembali
- Pembatasan ruang lingkup (Scoping)

Modul atau paket dapat diakses dengan menggunakan pernyataan **Import**.

Basic Module & Packages

How to Make Module

Misalnya, Anda telah membuat sebuah file bernama `mod.py` yang berisi kode berikut:

```
s = "Hacktiv8-PTP Python For Data Science"

a = [100, 200, 300]

def foo(arg):
    print(f'arg = {arg}')
```



```
class Foo:
    pass
```

Basic Module & Packages

How to Make Module

Anda dapat mengakses objek-objek pada file `mod.py` dengan mengimpor modul sebagai berikut:

```
import mod

print(mod.s)
print(mod.a)
mod.foo(['quux', 'corge', 'grault'])
x = mod.Foo()
print(x)
```

Basic Module & Packages

The Module Search Path

Jalur pencarian modul yang dihasilkan dapat diakses melalui variabel Python `sys.path`, yang diperoleh dari modul bernama `sys`:

```
import sys
sys.path
```

Dengan demikian, untuk memastikan modul Anda ditemukan, Anda perlu melakukan salah satu dari berikut ini:

- Tempatkan `mod.py` di direktori yang sama dengan skrip yang sedang dijalankan atau di direktori saat ini jika Anda menggunakan mode interaktif.
- Modifikasi variabel lingkungan `PYTHONPATH` untuk mencakup direktori tempat `mod.py` berada sebelum memulai interpreter.
- Atau, tempatkan `mod.py` di salah satu direktori yang sudah ada dalam variabel `PYTHONPATH`.
- Tempatkan `mod.py` di salah satu direktori yang bergantung pada instalasi, yang mungkin atau mungkin tidak dapat Anda akses untuk menulis, tergantung pada sistem operasi.

Basic Module & Packages

The Import Statement

Isi modul dapat diakses oleh pemanggil menggunakan pernyataan `import`. Untuk dapat diakses dalam konteks lokal, nama-nama objek yang didefinisikan dalam modul harus diawali dengan nama modul (misalnya `mod.<nama objek>`) :

```
import mod
print(mod.s)
```

Namun, bentuk alternatif dari pernyataan `import` memungkinkan objek-objek individu dari modul untuk diimpor langsung ke dalam tabel simbol pemanggil, sehingga Anda tidak perlu menyertakan nama modul setiap kali mengakses objek tersebut.

```
from mod import s, foo

print(s)
print(foo('quux'))
```


Basic Module & Packages

The `dir()` Function

Fungsi `dir()` dapat berguna untuk mengidentifikasi apa saja yang telah ditambahkan ke dalam ruang lingkup (namespace) oleh pernyataan `import`. Fungsi ini akan mengembalikan daftar nama-nama yang ada dalam ruang lingkup, termasuk variabel, fungsi, dan objek lainnya.

```
>>> from mod import a, Foo
>>> dir()

['Foo', '__annotations__', '__builtins__', '__doc__',
 '__loader__', '__name__', '__package__', '__spec__', 'a', 'mod']
```

Anda juga dapat mengimpor seluruh modul dengan nama alternatif:

```
>>> import mod
>>> dir(mod)

['Foo', '__builtins__', '__cached__', '__doc__', '__file__',
 '__loader__', '__name__', '__package__', '__spec__', 'a', 'foo', 's']
```

Basic Module & Packages

Python Packages

Paket memungkinkan struktur hierarkis untuk namespace modul menggunakan notasi titik (dot notation). Dengan cara yang sama seperti modul membantu menghindari bentrokan antara nama variabel global, paket membantu menghindari bentrokan antara nama modul.

Membuat paket cukup sederhana, karena memanfaatkan struktur file hierarkis yang ada di sistem operasi.

Sebagai contoh, ada sebuah direktori bernama `pkg` yang berisi dua modul, `mod1.py` dan `mod2.py`. Berikut adalah contoh isi dari modul-modul tersebut:



```
#mod1.py
def foo():
    print('[mod1] foo()')

class Foo:
    pass
```

```
#mod2.py
def bar():
    print('[mod2] bar()')

class Bar:
    pass
```

Basic Module & Packages

Python Packages

Dengan struktur sebelumnya, jika direktori `pkg` berada di lokasi yang dapat ditemukan (dalam salah satu direktori yang terdaftar di `sys.path`), Anda dapat merujuk ke kedua modul tersebut dengan notasi titik (dot notation) seperti `pkg.mod1` dan `pkg.mod2`, dan mengimpor mereka dengan sintaks yang sudah Anda kenal.

```
>>> import pkg.mod1, pkg.mod2
>>> pkg.mod1.foo()
[mod1] foo()

>>> from pkg.mod1 import foo
>>> foo()
[mod1] foo()
```

Basic Module & Packages

PIP

`pip` adalah pengelola paket untuk Python. Ini adalah alat yang memungkinkan Anda untuk menginstal dan mengelola pustaka tambahan serta dependensi yang tidak termasuk dalam pustaka standar Python.

Anda dapat mempelajari tentang perintah-perintah yang didukung oleh `pip` dengan menjalankannya dengan opsi `help`:

```
pip help
```

Seperti yang dapat Anda lihat, `pip` menyediakan perintah `install` untuk menginstal paket. Anda bisa menjalankan perintah ini untuk menginstal paket `requests`:

```
pip install requests
```

Selain itu, Anda dapat menggunakan perintah `list` untuk melihat paket-paket yang terpasang di lingkungan Python Anda:

```
pip list
```



External References

Colab Link

[Visit Here](#)



Hacktiv8id



Hacktiv8id



Hacktiv8



Hacktiv8 Indonesia